AUTONOMOUS AGENTS

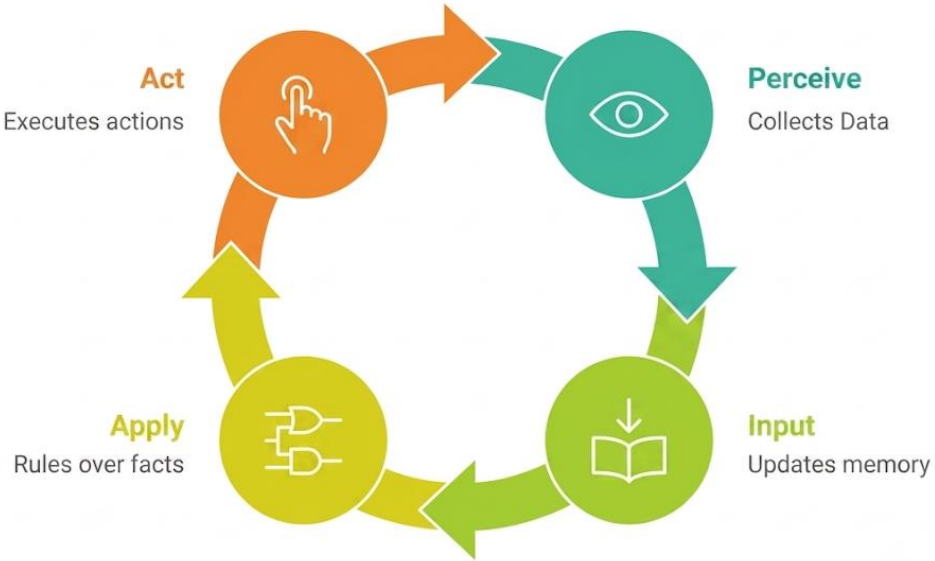
Introduction: Agents

Reactive (Rule-Based) Agent

- Executes predefined rules
- No learning or reasoning
- Behavior is entirely programmed

LLM-Powered Agent

- Uses a language model for decision making
- Can reason, plan, and use tools
- Behavior emerges from learned knowledge rather than hard-coded rules



IF/THEN DECISION LOOPS (REACTIVE (RULE-BASED) AGENTS)

- **A reactive agent:**
  - Observes the environment
  - Applies predefined rules
  - Produces a fixed action
- **Characteristics**
  - No memory
  - No learning
  - No reasoning
  - No understanding of context

EXAMPLE: THERMOSTAT AGENT

Input:

- Current temperature
- Target temperature

Output:

- Heat
- Cool
- Idle

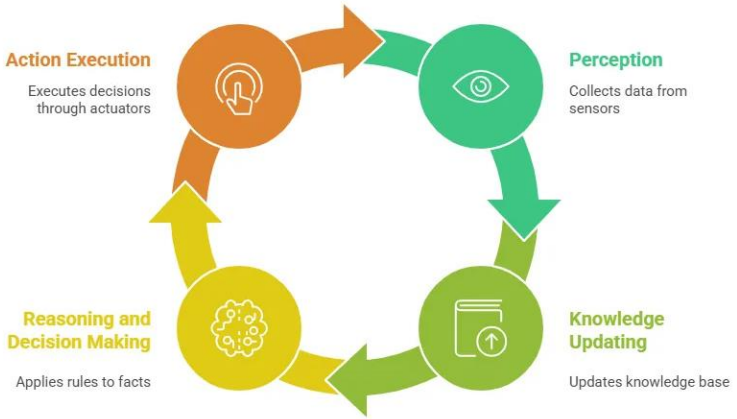
```
if current_temp < target_temp - 1 > HEAT_ON

elif current_temp > target_temp + 1 > COOL_ON

else > IDLE
```

ANOTHER EXAMPLE: CUSTOMER SUPPORT AGENT

- User message →
  - Contains "**refund**" → Billing Team
  - Contains "**password**" or "**login**" → Account Team
  - Contains "**broken**" or "**not working**" → Technical Team
  - Otherwise → General Queue
- What If the customer writes: "I want my money back."?
- The system may fail because that exact phrase wasn't programmed.

PROS AND CONS OF RULE-BASED AGENTS

Advantages	Limitations
Deterministic	No generalization
Fast	Cannot understand unseen phrasing
Easy to debug	Rule complexity grows rapidly
Predictable behavior	Brittle in real-world scenarios



LLM-POWERED AGENTS

- They combine:
  - **Large Language Model (LLM)**
  - **Reasoning loop**
  - **Memory**
  - **External tools**
  - **Decision making**
- Instead of following fixed rules, the agent decides:
  - What should I do?
  - Which tool should I use?
  - What should I do next?

WHAT IS A LARGE LANGUAGE MODEL?

- A Large Language Model (LLM) is fundamentally a next-token predictor.
- Given a sequence of tokens, predict the probability
  distribution of the next token.
- Temperature determines how to sample from this probability
- Everything that looks like reasoning or planning emerges
  from repeated next-token prediction.

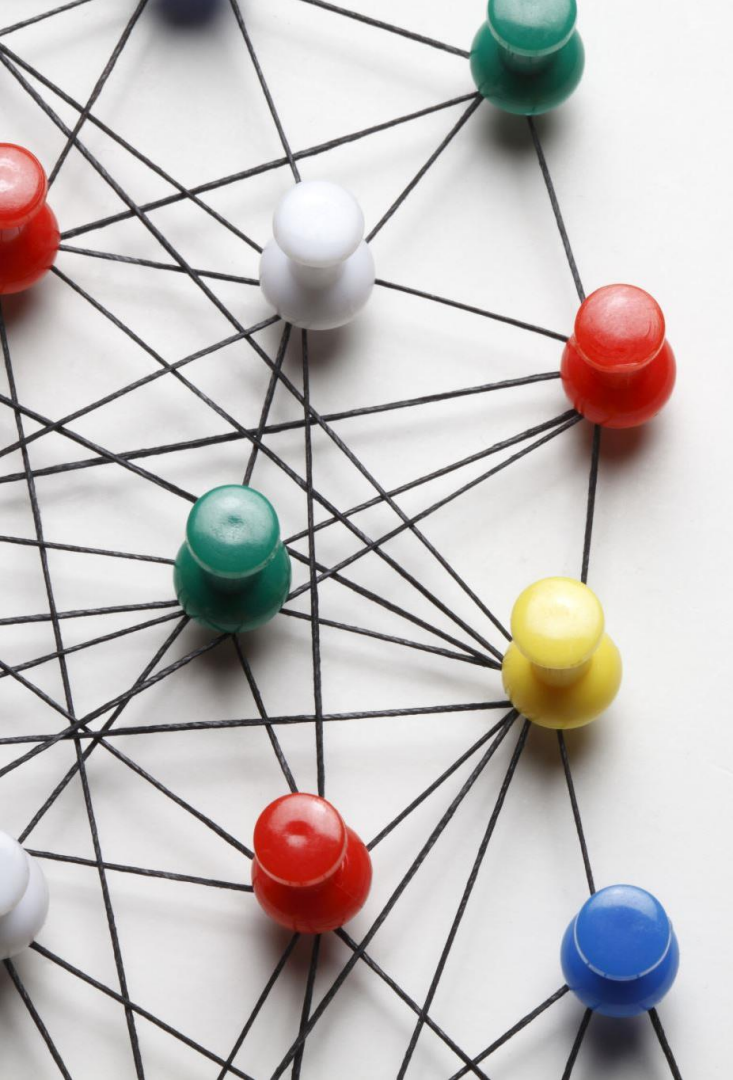
Enter text:

One, two,

3198 11 734 11

Prediction

#	probs	next token ID	predicted next token
0	39.71%	1115	three
1	16.97%	290	and
2	7.55%	734	two
3	3.76%	1440	four
4	2.76%	393	or
5	2.18%	1936	five
6	1.57%	530	one
7	1.43%	345	you
8	1.15%	257	a
9	0.84%	3598	seven

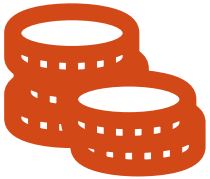


NEXT-TOKEN PREDICTION IN PRACTICE

- **Input**
  - "The capital of France is"
- **Possible next tokens**
  - " Paris" → 91%
  - " located" → 3%
  - Generation repeats: predict → append → predict
- **The result is stochastic and sequential.**

THE TRANSFORMER, BRIEFLY

Tokenization: Text is split into tokens and mapped to vectors.



Self-Attention: Each token weighs which other tokens in the context matter.

Stacked Layers: Repeated blocks refine syntax into task-relevant patterns.

CONSEQUENCES FOR AGENT BUILDERS

No persistent internal state

- Every API call is stateless; memory must be supplied again.

No guaranteed output structure

- Without constraints, the model may return prose, malformed JSON, or invalid tool inputs.

Errors compound

- Each new token is conditioned on earlier output—including earlier mistakes.

Intentional Language

- The model decided to check the balance.
- It noticed the result was wrong.
- It wants to loop again.

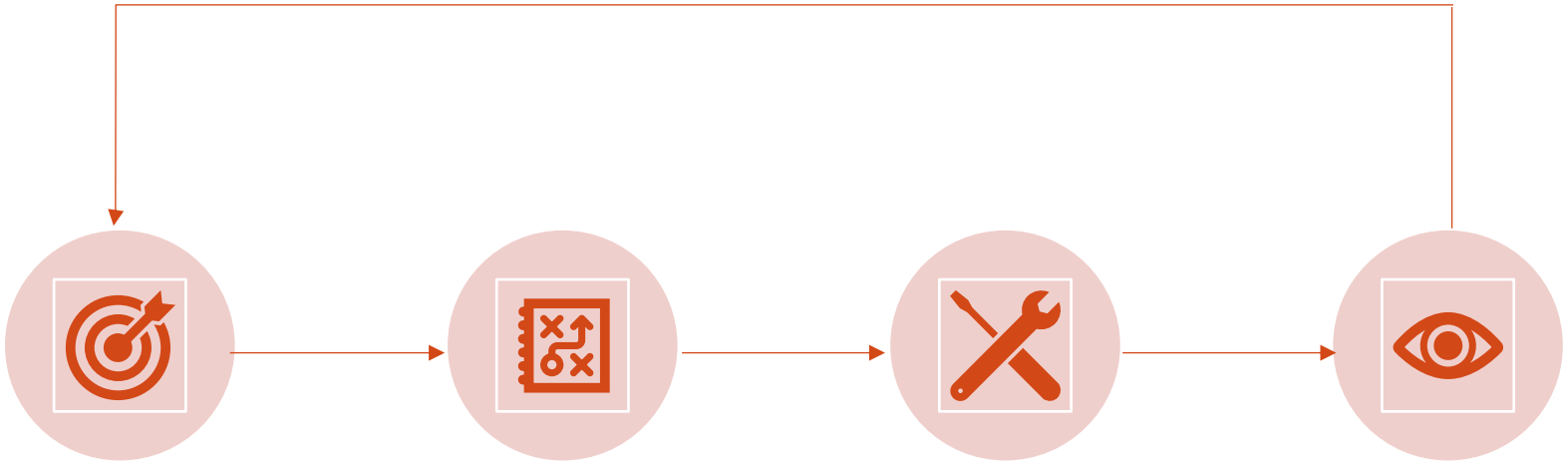
Mechanistic Reality

- Each call starts from the supplied context.
- The model predicts a next-token distribution.
- There is no persistent state of mind between calls.

Debug the context, not the agent's intent

- **Weak framing**
  - Why did the agent decide the refund was invalid?
- **Better framing**
  - Was the order data present, correct, and unambiguous?
  - Did a malformed observation remain in context?
  - Was important context truncated, summarized, or diluted?
- **Debug the tokens available at generation time.**

THE AGENT LIFECYCLE



INTENT: UNDERSTAND THE GOAL

PLAN: CHOOSE THE NEXT ACTION

EXECUTE: CALL A TOOL OR API

OBSERVE: CHECK RESULT OR TERMINATE

Loop back when the goal is not yet satisfied

UNCONSTRAINED REACT

Thought: I need the account balance.

Action: get_account_balance(user_id="1234")

Observation: {"balance": 42.50}

Thought: I should check the requested refund...

Action: ...

The model controls the tool choice, result interpretation, and stopping point.

High latency:

- A simple lookup can become many sequential LLM round-trips.

Cascading errors:

- A hallucinated observation becomes context for every later step.

Infinite or near-infinite loops:

- Without a budget or termination rule, stopping is only a heuristic.

TOKEN ECONOMICS: THE RE-TRANSMISSION TAX

- **Every model call is stateless**
  - Each step usually re-sends all prior thoughts, actions, and observations.
- **Context grows with every step**
  - Step N repays for much of the context from steps 1 through N−1.
- **Cost grows faster than the step count**
  - Long reasoning chains can approach quadratic cumulative token growth.

WORKED COST COMPARISON

Approach	LLM Calls	Approx. Tokens	Latency
Unconstrained ReAct	~12	35,000–45,000	8–15 sec
Deterministic Routing	1–2	500–1,500	<1 sec

The exact numbers vary; the compounding shape is the important part.

DETERMINISTIC ROUTING

Use the model only to classify a request, then follow fixed code paths.

Push control flow out of the model and into code.

EXAMPLE: INTENT CLASSIFICATION

The model chooses one label from a fixed set.

```
intent = classify(user_input)
if intent == REFUND:
    return run_refund_workflow()
elif intent == ACCOUNT:
    return run_account_workflow()
elif intent == TECHNICAL:
    return run_technical_workflow()
else:
    return route_to_human()
```

Everything after classification is ordinary, testable code.

WHY DETERMINISTIC ROUTING WORKS

- One LLM call instead of many
- Bounded, predictable latency and cost
- A wrong classification causes one misroute—not a cascading trajectory
- Downstream workflows are fully testable
- Decisions remain easy to trace and debug

WHEN ROUTING IS NOT ENOUGH

Deterministic Routing

- The action set is small and known.
- The request maps directly to one workflow.
- No step depends on a tool result.

Multi-Step Reasoning

- The next action depends on the previous result.
- The agent must adapt during execution.
- Example: look up an order, then refund or escalate.

CONSTRAINED REACT

• KEEP THE REASONING-AND-ACTING LOOP

• VALIDATE EVERY STEP AGAINST A SCHEMA

• RESTRICT ACTIONS TO AN ALLOW-LIST

• SET A HARD MAXIMUM NUMBER OF STEPS

• MAKE TOOL ERRORS AND ESCALATION EXPLICIT

• GUARANTEE A FINAL ANSWER OR A CONTROLLED HANDOFF

STRUCTURED OUTPUT AT EVERY STEP

Replace free-text actions with a validated schema.

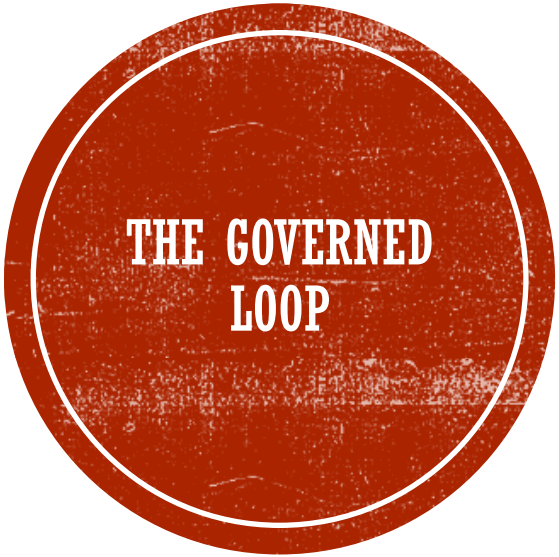
```
class AgentStep(BaseModel):
    thought: str
    action: Literal["get_account_balance",
                    "process_refund",
                    "escalate",
                    "final_answer"]
    action_input: dict
    is_final: bool
```

Malformed output is rejected before it reaches a tool.

Reliable structured output in practice

- **Prompted JSON — weakest**
  - A soft instruction can still produce code fences, preambles, or malformed JSON.
- **Native structured output — better**
  - The API constrains generation to a JSON schema where supported.
- **Validate and retry — always**
  - Schema-valid JSON still needs semantic checks and a bounded retry policy.

- Validate the AgentStep schema
- Reject actions outside ALLOWED_ACTIONS
- Validate parameters against the selected tool
- Return the exact validation error to the model
- •Retry at most twice
- Escalate or fail explicitly when retries are exhausted
- **Valid JSON is necessary—not sufficient.**



MAX_STEPS = 6

A hard termination budget

Generate	Generate one structured AgentStep
Validate	Validate the action and parameters
Execute	Execute only allow-listed tools
Feed back	Feed back an explicit result or error
Stop	Stop, or escalate when the budget is exhausted

UNCONSTRAINED VS. CONSTRAINED REACT

Unconstrained Failure	Constrained Mitigation
Unbounded reasoning chains	Hard MAX_STEPS budget
Unvalidated model output	Schema validation + action allow-list
No termination guarantee	Explicit final flag + escalation path

CHOOSING THE RIGHT PATTERN

1. Any ambiguity?
Is there unstructured input to interpret?

No

Plain Code
Do not add a model call

Yes

2. Are next actions known?
Small, fixed, and independent of later results?

Yes

Routing
Classify once

No

Constrained ReAct
Adapt after observations

A WORKED DECISION TABLE

Task	Ambiguous?	Depends on Result?	Pattern
Cancel Order button	No	—	Plain code
I want my money back	Yes	No	Routing
Find why my order is late	Yes	Yes	Constrained ReAct
Parse a webhook payload	No	—	Plain code
Something is wrong with my bill	Yes	No	Routing

CHOOSE THE LEAST POWERFUL PATTERN

Move right only when the task genuinely requires more model flexibility.

Use only as much model freedom as the task actually requires.

DISCUSSION / LAB QUESTIONS

1. What is the smallest set of inputs that breaks the rule-based support agent? Would a deterministic router handle them?

2. If execute_tool calls another LLM internally, how should that affect the MAX_STEPS budget?

3. Name one task suited to deterministic routing and one that requires step-dependent reasoning. What distinguishes them?

4. Rewrite "the agent forgot" as a mechanistic context-window explanation.

5. Classify three support tickets using the decision framework before writing code. (I will write on board)

6. Log token counts per ReAct step and plot cumulative tokens against step number. (at home)

7. Remove the action allow-list and try to provoke a hallucinated tool name. (at home)